

Badger Code

v0.5.3 Canary Bulgular■ & Bütçe Yol Haritas■

Terminal-Bench 2.1 Coding Agent Scaffold — Kaggle MLM26 EfficientCoder

Tarih: 2026-09-24

1. Genel Özet — Sürüm Zinciri

Bu rapor, v0.5.1 → v0.5.2 → v0.5.3 arasındaki scaffold düzeltmelerinin özetini, v0.5.3 canary koşusunun (6 run, 24 trial) derin analiz bulgularını ve bütçe/yol haritası kararlarını tek bir belgede toplar.

Sürüm	Ana Değişiklik	Sonuç (özet)
v0.5 baseline	Doğru dataset (Terminal-Bench 2.1) ile ilk taban	3/24 (%12.5)
v0.5.1	PATH-kalıcılık notu, toplu-script notu, LLM_MAX_TOKENS limitleri (11.1)	2/18 (%11.1)
v0.5.2	write_file blind-spot fix, regex/Python notu, python3-path notu	3/21 (%14.3), sqlite-with-gcov ilk kez geçti
v0.5.3	write_file append modu + append-guard (chunked-write)	24 trial, kararsız sonuç — bkz. Bölüm 2

2. v0.5.3 Canary Derin Analiz Bulguları

Koşum: ilk 3 run (7 görev) arasında **APIConnectionError/AgentTimeoutError** kirliliğine uğradı (Nebius altyapı sorunu). Bütçeyi korumak için run4–6, v0.5.3'ün hedeflediği mekanizmayla doğrudan ilgili 4 göreve daraltıldı: regex-log, build-cython-ext, chess-best-move, log-summary-date-ranges.

2.1 — log-summary-date-ranges: 3/6 PASS (büyük kazanım, ama append'le ilgisiz)

PASS olan 3 trial'ın hiçbirinde **append=true** çalışmadı (0 çalıştı). Kazanan mekanizma v0.5.1'in "5+ dosya varsa toplu script yaz" notu — model /app/analyze_logs.py'yi tek seferde (append=false) yazıp çalıştırdı. v0.5.2'yi kirleten verifier DNS hatası bu turda oluşmadığı için kazanım ilk kez görünür oldu.

- Run 1 FAIL: toplu script yazmadı, CLI find/awk ile satır satır hesaplamaya çalışıp stuck_loop_detected ile öldü.
- Run 5 FAIL: script yazdı ama regex yerine "if severity in line" kullanıp gövde metnindeki kelimeleri de saydı (saf model mantık hatası).
- Run 6 FAIL: script yazmadı, read_file/find/grep ile tarayıp döngüye girdi.

2.2 — chess-best-move: append fix çalışmıyor (model talimat görmezden geliyor)

En önemli metodolojik ders bu görevden çıktı. Scaffold TAM istenen şekilde çalıştı: model 4096 token'da kesilince (Run 5, Turn 12), güçlendirilmiş kurtarma mesajı doğrudan enjekte edildi:

```
"Note: Writing to '/app/chess_move.py' appears too large... Split the file into smaller chunks: write first chunk with append=false, then append subsequent chunks using append=true"
```

Model bu uyarıyı tamamen görmezden geldi, aynı monolitik dosyayı tekrar yazmaya çalıştı, 872 saniye sonra Nebius bağlantısı koptu. **Genel ilke:** bir prompt-tabanlı kurtarma mekanizmasının doğrudan çalıştırılmasını kanıtlamak (mesaj doğrudan turda doğru içerikle gönderiliyor) onun etkili olduğunu kanıtlamaz — model mesajı okuyup uyup uymaması ayrı bir deşifren, ayrı ölçülmeli. Diğer trial'larda model tahtayı görsel olarak okuyamadığı için (Kategori C, model yetkinlik sınırı) hamle uydurdu; doğrulama kapısı bunları reddetti.

2.3 — regex-log: döngü fix'i çalıştı, ama YENİ bir hastalık ortaya çıktı ()

write_file blind-spot fix'i (v0.5.2) dođurulandđ: eskiden 95 tur süren döngü artđk 6–20 turda erken yakalanđyor. Ama model artđk TEK bir turda devasa patolojik regex/analiz metni üretilip (Run 3: yüzlerce negatif lookahead unroll ederek 4096 token'đ doldurup JSON'đ kđrdđ; Run 1: 11 tur ardđđđk 9K karakter; Run 5: 12.5K karakter) araç çađđđramadan kesiliyor — bu 900s AgentTimeoutError'a veya Nebius socket kopmasđna yol açđyor.

Genel ilke: "tekrarlayan-tur" döngüsü ile "tek-tur bođulmasđ" farklı hastalđklardđr, aynı semptomu (timeout) üretebilirler. Bir fix'in semptomu düzeltmemesi, fix'in bađarđsđz olduđu anlamđna gelmeyebilir — burada fix iđe yaradı, altđndaki farklı bir sorun görünür oldu.

2.4 — build-cython-ext: yeni, somut, kodlanabilir aday (đ)

python3-path notu okundu ve etkili oldu — model pip ile cython/setuptools kurdu, uzantđlarđ derledi (build_ext --inplace, exit 0). Ama 5/5 gerçek trial'da hâlâ FAIL:

- Kök neden 1: model asla "pip install ." çađđđđđrmđyor — sadece --inplace derliyor, paket aktif ortama kurulmuyor, verifier "import pyknotid" arayđınca ModuleNotFoundError alđyor.
- Kök neden 2: pyknotid kütüphanesinin kendi kodu ("from fractions import gcd") Python 3.13'te kaldđrđlan bir fonksiyonu çađđđđyor — bu kütüphanenin kendi uyumsuzluđu, Kategori C, dokunulmuyor.

Kodlanabilir aday (henüz yapılmadı, karar bekliyor): genel prompt notu — "bir Python eklentisi derledikten sonra, görev özellikle in-place derleme istemiyorsa pip install . ile aktif ortama kur."

2.5 — Özet tablo

Görev	v0.5.2	v0.5.3	Deđiđim	Kök neden
log-summary-date-ranges	0/3 (DNS hatası)	3/6 PASS (%50)	đ Büyük sđçrama	Toplu script kuralđ kazandırdđ, append kullanđlmadı
chess-best-move	0/3	0/6	đ Deđiđmedi	Model append uyarđsđnđ dinlemiyor; asđl sđnđr görsel ar
regex-log	0/3	0/6	đ Farklı hastalđk	95-tur döngü çözüldü ama tek-tur token bođulmasđ (4096)
build-cython-ext	0/3	0/6	đ Kurulum eksik	Derleme OK ama pip install . yok + pyknotid'in Python 3.13 u

3. Bütçe Yol Haritası

3.1 — Bütçe durumu

Kalem	Tutar
Harcanan (v0.4 → v0.5.3, tüm iterasyonlar)	\$14
Kalan (şimdi)	\$11
Gelecek ay eklenecek kredi	\$25
Toplam garanti bütçe (kalan)	\$36

3.2 — Yarınma kuralı kontrolü: k=1 yeterli

RULES.md ve WRITEUP_TEMPLATE.md incelendi — pass@k veya ortalama-N-konu diye bir mekanizma YOK. Skor tek bir Harbor konusundan hesaplanıyor:

```
tb_score = 89 görevin TEK konudaki ortalama reward'ın (0-1)
total_tokens = n_input_tokens + n_output_tokens (result.json)
leaderboard_score = tb_score - 0.01 x (total_tokens / 1,000,000)
```

Bu, token optimizasyonumuzun (4096 limit, cyclic-loop erken durdurma) sadece \$ maliyeti için değil, doğrudan leaderboard skoru için de kritik olduğunu doğruluyor.

3.3 — Üç fazlı plan

Faz	Ne	Amaç	Durum
A	Canary iterasyon döngüsü (7 görev, n=3 veya daha fazla) için n=1 bulup düzeltmek	Scalability setinin bulup düzeltmek	ŞU AN BURADAYIZ
B	TEK 89-görev, k=1, n=1 "scalability kontrolü" kolunu	Skor değil — 89 ölçeğinde altyapı sürprizi yaşıyor	Beklenen + gerçek \$/trial ölçmek
C	Final/skorlanan konu	Submission card için tb_score/total_tokens/leaderboard_score hesabı	Deneyin sonucu — B başlıyor

3.4 — Dürüstlük ilkesi (kullanıcı karar, değilmez)

Faz B sonrası bütçe Faz C'ye (+varsa buffer tekrar) yetmiyorsa, riske girip yarın bir tam konuya para yakmak YERİNE "buraya kadar geldik, kalanına param yetmedi" diyerek son sağlıklı checkpoint'te (muhtemelen v0.5.x serisinin son hali) dürüstçe durulacak.

3.5 — Canary metodolojisi notu

n=1, 7 görevlik canary setinde güvenilirmez oldu bu projede defalarca kanıtlandı (configure-git-webserver'n 3/3 → 1/3 → 2/3 salınması gibi) — n=3 canary için hâlâ gerekli. Ama 89 görevlik final konu farklı: 89 görev zaten kendi içinde 7 görevden istatistiksel olarak çok daha stabil bir örneklem, o yüzden final konunun k=1 olması metodolojik bir gerileme DEĞİL.

4. Şıradaki Adım — Açık Karar

Kullanıcı karar bekliyor: build-cython-ext'in "pip install ." notunu (küçük/ucuz fix) kodlayıp v0.5.4'e mi geçelim, yoksa burada durup doğrudan Faz B'ye (89 görevlik scalability konusu) mi geçelim?

Kaynaklar: docs/plan.md (2026-09-24 bölümü), docs/worker1-agy-v053-canary-deepanalysis.md,
~/Desktop/agent-brain/wiki/badger-code-coding-agent-scaffold-bulgulari.md